

System Design Write-Up

Healthcare Appointment & Follow-up Manager

1. Double-Booking Prevention

The system prevents two patients from ever holding the same doctor slot through both a database constraint and an application-level transaction. The Appointment table enforces a unique composite constraint on (doctorId, slotStart), so even if two booking requests arrive simultaneously, the database rejects the second insert — a safety net no application logic can bypass.

The booking API wraps the "check availability, then create" sequence inside a single transaction, closing the read-then-write race where two requests could both read a slot as free within milliseconds of each other. If the insert fails due to the unique constraint, the API catches that error and returns a 409 Conflict with a clear message, prompting the patient to pick another time instead of silently creating a duplicate booking.

2. Slot Hold Mechanism

Booking is a multi-step flow — select a slot, fill a symptom form, then confirm — so the slot must be protected mid-flow without permanently locking it if abandoned. Selecting a slot creates an Appointment immediately in a HELD state with a holdExpiresAt timestamp five minutes out.

A slot counts as available if no appointment exists for it, or if an existing HELD appointment's hold has expired. Abandoned bookings self-heal after five minutes; a scheduled job also periodically clears stale HELD rows. On confirmation, the row moves to CONFIRMED and the hold timestamp is cleared, becoming a permanent booking protected by the constraint above.

3. Doctor Leave Conflict Handling

Doctors can be marked on leave even after patients have booked slots on that date. This is treated as a cascading event, not a silent flag. When a leave record is created, the backend immediately queries every CONFIRMED or HELD appointment for that doctor on that date, transitions each to CANCELLED, and creates a notification for the affected patient explaining the doctor's unavailability and inviting a rebooking.

The same process removes the associated Google Calendar event so both calendars stay accurate. Because this cascade runs inside the same request that creates the leave record, there is no window where a doctor is on leave while patients remain unaware their appointment is no longer valid.

4. Notification Failure Handling

Email and calendar sync can fail for reasons outside the app's control — provider outages, rate limits, network issues. These failures never block the core booking flow; a patient can always book, cancel, or receive care regardless of whether an email sends on the first try.

Every outbound notification is logged with a RETRYING status before the send is attempted. The send itself is wrapped in try/catch: success marks it SENT, failure stores the error and increments an attempt count. A background job re-scans RETRYING records and retries any under a maximum attempt threshold, after which they are marked FAILED for manual review. This gives an auditable trail of every communication and lets transient failures self-correct without user-facing disruption.

5. LLM Reliability

Pre-visit and post-visit AI summaries follow the same philosophy. Each LLM call is wrapped in try/catch with a request timeout, so a slow model call cannot hang the flow. On failure, a clear fallback message replaces the summary, directing the doctor or patient to the raw symptoms or clinical notes instead — an LLM outage degrades gracefully rather than blocking care.

6. Trade-offs Given the Timeline

A few pragmatic simplifications were made under the project deadline and are stated here explicitly. SQLite is used instead of a managed Postgres instance for speed of setup; production would move to Postgres for better concurrent-write handling. Scheduled jobs run through a cron-triggered API route rather than a dedicated queue such as Bull with Redis, which would be more robust under real load. Google Calendar integration is scoped to the doctor's calendar plus an emailed invite for the patient, avoiding the setup overhead of per-user OAuth consent. None of these choices compromise the correctness of the booking, conflict-handling, or notification logic described above — they affect only operational robustness at larger scale.